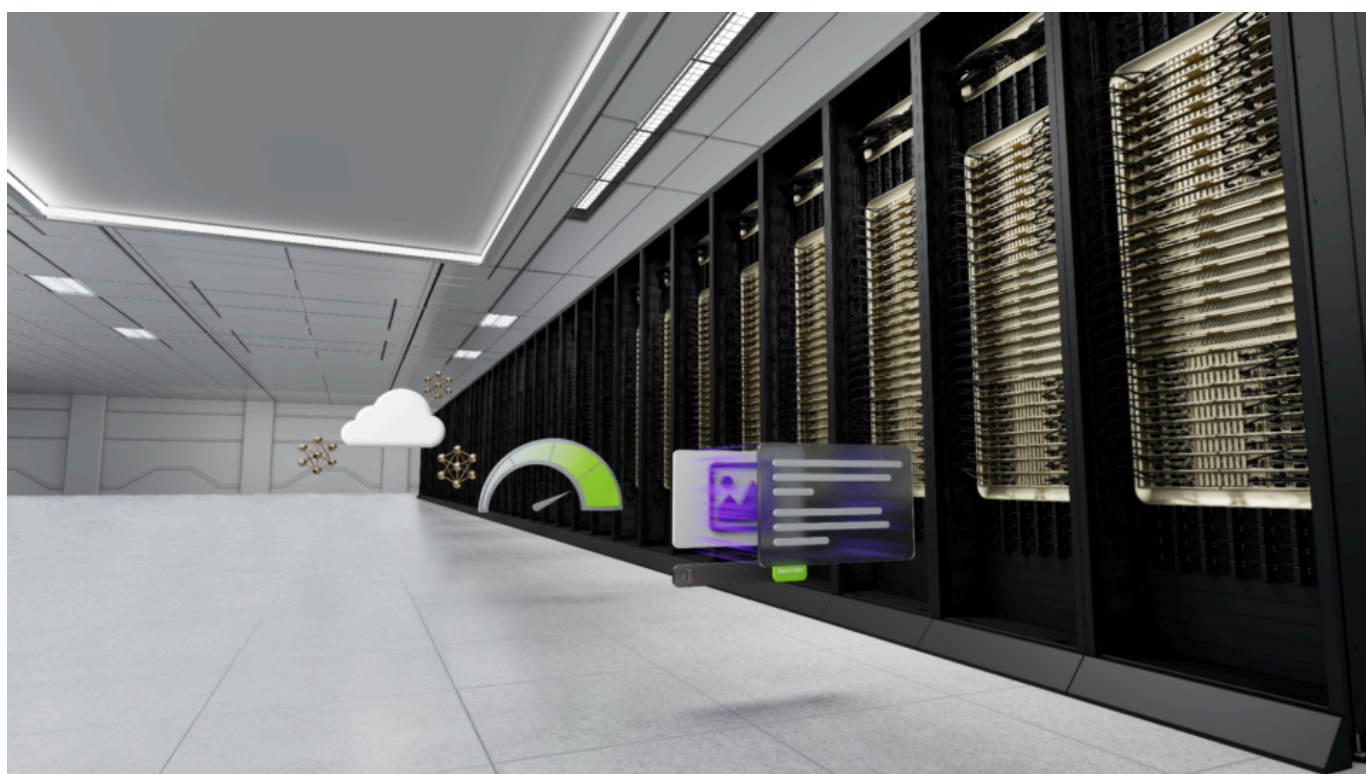


LLM Inference Benchmarking: Fundamental Concepts



Apr 02, 2025

 +29 Like  Discuss (0)

By [Vinh Nguyen](#), [Wenwen Gao](#), [Emily Apsey](#), [Ganesh Kudleppanavar](#), [Neelay Shah](#) and [Elias Bermudez](#)



AI-Generated Summary



This is the first post in the large language model latency-throughput benchmarking series, which aims to instruct developers on common metrics used for LLM benchmarking,

The past few years have witnessed the rise in popularity of [generative AI](#) and [large language models \(LLMs\)](#), as part of a broad AI revolution. As LLM-based applications are rolled out across enterprises, there is a need to determine the cost efficiency of different AI serving solutions. The cost of an LLM application deployment depends on how many queries it can process per second while being responsive to end users and supporting an acceptable level of response accuracy. This post focuses specifically on LLM throughput and latency measurement as part of assessing LLM application costs.

NVIDIA empowers developers with full-stack innovations, spanning chips, systems, and software. The NVIDIA inference software stack includes [NVIDIA Dynamo](#), [NVIDIA TensorRT-LLM](#), and [NVIDIA NIM](#) microservices. To support developers with benchmarking inference performance, NVIDIA also offers [GenAI-Perf](#), an open-source generative AI benchmarking tool. Learn more about [using GenAI-Perf to benchmark](#).

Evaluating the performance of LLMs can be accomplished using a variety of tools. These client-side tools offer specific metrics for LLM-based applications but differ in how they define, measure, and calculate different metrics. This can be confusing and can make it difficult for results from one tool to be compared with the results of another.

In this post, we clarify the common metrics and the subtle differences in how popular benchmarking tools define and measure these metrics. We also discuss the important parameters for benchmarking.

Load testing and performance benchmarking

Load testing and performance benchmarking are two distinct approaches to evaluating the deployment of an LLM. Load testing focuses on simulating a large number of concurrent requests to a model to assess its ability to handle real-world traffic at scale. This type of testing helps identify issues related to server capacity, autoscaling tactics, network latency, and resource utilization.

throughput, latency, and token-level metrics. This type of testing helps identify issues related to model efficiency, optimization, and configuration.

While load testing is essential for ensuring the model can handle a large volume of requests, performance benchmarking is crucial for understanding the model's ability to process requests efficiently. By combining both approaches, developers can gain a comprehensive understanding of their LLM deployment capabilities and identify areas for improvement.

How LLM inference works

Prior to examining benchmark metrics, it is important to understand how LLM inference works, and to become familiar with related terminology. An LLM application produces results through inference stages. For a given specific LLM application, these stages include:

- **Prompt:** User provides a query
- **Queuing:** Query joins the queue for processing
- **Prefill:** The LLM model processes the prompt
- **Generation:** The LLM model outputs a response, one token at a time

An AI token is a concept specific to LLMs and is core to LLM inference performance metrics. It is the unit, or smallest lingual entity, that LLMs use to break down and process natural language. The collection of all tokens is known as a vocabulary. Each LLM has its own tokenizer that is learned from the data so as to represent the input text efficiently. As an approximation, for many popular LLMs, each token is ~0.75 English words.

Sequence length is the length of the sequence of data. The *Input Sequence Length (ISL)* is how many tokens that the LLM gets. It includes the user query, any system prompt (instructions for the model, for example), previous chat history, chain of thought (CoT) reasoning, and documents from the retrieval-augmented generation (RAG) pipeline. The *Output Sequence Length (OSL)* is how many tokens the LLM generates. *Context length* is how many tokens the LLM uses at each generation step, including both the input and output tokens generated to that point. Each LLM has a maximum context length that can

Streaming is an option that allows partial LLM outputs to be streamed back to users in the form of chunks of tokens generated incrementally. This is important for chatbot applications, where it is desirable to receive an initial response quickly. While the user digests the partial content, the next chunk of the result arrives in the background. In contrast, in nonstreaming mode, the full answer is returned all at once.

LLM inference metrics

This section explains some of the common metrics used in the industry, including time to first token and intertoken latency, as shown in Figure 1. Although they seem straightforward, there are some slight but significant differences between various benchmarking tools.

Figure 1. LLM inference performance metrics

Time to first token

model's output.

Note that both GenAI-Perf and LLMPerf benchmarking tools disregard the initial responses that have no content or content with an empty string (no token present). This is because the TTFT measurement is meaningless when the first response has no token in it.

Figure 2. The process leading to the generation of the first token

TTFT generally includes both request queuing time, prefill time, and network latency. The longer the prompt, the larger the TTFT. This is because the attention mechanism requires the whole input sequence to compute and create the so-called key-value (KV) cache, from which point the iterative generation loop can begin. Additionally, a production application can have several requests in progress, so the prefill phase of one request may overlap with the generation phase of another request.

End-to-end request latency

End-to-end request latency (e2e_latency) indicates the time it takes from submitting a query to receiving the full response, including the time for queueing and batching and network latencies (Figure 3). Note that in streaming mode, the detokenization step can be done multiple times when partial results are returned to the user.

Figure 3. End-to-end request latency

For an individual request, the end-to-end request latency is the time difference between when the request is sent and the final token is received:

$$e2e_latency = TTFT + generation_time$$

Note that `generation_time` is the duration from when the first token is received to when the final token is received (Figure 1). In addition, GenAI-Perf removes the last (done) signal or empty response, so these aren't included in the `e2e_latency`.

Intertoken latency

Intertoken latency (ITL) is the average time between the generation of consecutive tokens in a sequence. It is also known as time per output token (TPOT).

Although this seems to be a straightforward definition, there are some intricate differences in how the metric is collected through the different benchmarking tools. For example, GenAI-Perf does not include TTFT in the average calculation (as opposed to LLMPerf, which does include the TTFT).

GenAI-Perf defines ITL with the following equation:

$$ITL = \frac{e2e_latency - TTFT}{Total_output_tokens - 1}$$

The equation used for this metric does not include the first token (hence subtracting 1 in the denominator). This is done so that ITL is a characteristic of the decoding part of the request processing only.

It's important to note that with longer output sequences the KV cache grows, so the memory cost also grows. The cost of attention computation grows as well: for each new token, this cost is linear in the length of the input plus output sequence generated so far. However, this computation is generally not compute-bound. Consistent ITLs signify efficient memory management and better memory bandwidth as well as efficient attention computation.

Tokens per second

Tokens per second (TPS) per system represents the total output tokens per second throughput, accounting for all the requests happening simultaneously. As the number of requests increases, the total TPS per system will increase, until it reaches a saturation point for all the available GPU compute resources, beyond which it will possibly decrease.

For the example shown in Figure 5, assume the timeline of the entire benchmark with n total requests. Events are defined as follows:

- **L_i** : End-to-end latency of i -th request
- **T_{start}** : Start of benchmark
- **T_x** : Timestamp of the first request
- **T_y** : Timestamp of the last response of the last request
- **T_{end}** : End of benchmark

Figure 5. Timeline of events in a benchmarking run

GenAI-Perf defines the TPS as total output tokens divided by the end-to-end latency between the first request and the last response of the last request:

$$TPS = \frac{\text{Total_output_tokens}}{T_y - T_x}$$

LLMPerf defines TPS as the total output tokens divided by the entire benchmark duration:

$$\frac{\text{Total_output_tokens}}{T_{\text{end}} - T_{\text{start}}}$$

As such, LLM-perf also includes the following overheads in the metric:

- Input prompt generation
- Request preparation
- Storing the responses.

In our observation, these overheads in the single concurrency scenario can sometimes account for 33% of the entire benchmark duration.

Note that the TPS calculation is done in a batch fashion and is not a live running metric. In addition, GenAI-Perf uses a sliding window technique to find stable measurements. This means that the given measurements will be from a representative subset of the fully completed requests, meaning the “warming up” and “cooling down” requests are not included when calculating the metrics.

TPS per user represents throughput from a single user perspective, and is defined as:

$$TPS_{\text{per user}} = \frac{\text{Output sequence length}}{e2e_latency}$$

This definition is for each user’s request, which asymptotically approaches 1/ITL as the output sequence length increases. Note that as the number of concurrent requests

Requests per second

Requests per second (RPS) is the average number of requests that can be successfully completed by the system in a 1-second period. It is calculated as follows:

$$RPS = \frac{\text{total_completed_requests}}{T_y - T_x}$$

Benchmarking parameters and best practices

This section presents some important test parameters and their sweep range, which ensures meaningful benchmarking and quality assurance.

Application use cases and their impact on LLM performance

An application's specific use cases will influence the sequence lengths (ISL and OSL), which will in turn impact how fast a system digests the input to form KV-cache and generate output tokens. A longer ISL will increase the memory requirement for the prefill stage and thus increase the TTFT. A longer OSL will increase the memory requirement (both bandwidth and capacity) for the generation stage and thus increase ITL. It is important to understand the distribution of inputs and outputs in your LLM deployment to best optimize your hardware utilization.

Common use cases and the likely ISL/OSL pairs include:

- **Translation:** Includes translation between languages and code and is characterized by having similar ISL and OSL of roughly 500~2000 tokens each.
- **Generation:** Includes generation of code, story, and email content and generic content through search. This is characterized by having an OSL of O(1,000) tokens, much longer than an ISL of O(100) tokens.

- **Reasoning:** Recent reasoning models generate a large number of output tokens in an explicit chain-of-thought, self-reflection-and-verification reasoning approach to solve complex problems, like coding, maths or puzzles. This is characterized by short ISL of $O(100)$ tokens and a large OSL of $O(1000-10000)$ tokens.

Load control parameters

Load control parameters as defined in this section are used to induce loads on LLM systems.

Concurrency N is the number of concurrent users, each having one active request, or equivalently the number of requests concurrently being served by an LLM service. As soon as each user's request receives a complete response, another request is sent to ensure that at any time the system has exactly N requests. Concurrency is most frequently used to describe and control the load induced on the inference system.

Note that LLMPerf sends out requests in batches of N requests, but there is a draining period where it waits for all the requests to complete before sending out the next batch. As such, towards the end of the batch, the number of concurrent requests reduces gradually to 0. This differs from GenAI-Perf, which always ensures N active requests throughout the benchmarking period.

The *maximum batch size* parameter defines the maximum number of requests that the inference engine can process simultaneously, where *batch* is the group of simultaneous requests being processed by the inference engine. This may be a subset of the concurrent requests.

If the concurrency exceeds the maximum batch size multiplied by the number of active replicas, some requests will have to wait in a queue for later processing. In this case, you may see an increase in TTFT value due to the queueing effect of waiting for a slot to open up.

Request rate is another parameter that can be used to control load by determining the rate at which new requests are sent. Using a constant (or static) request rate r means 1 request

GenAI-Perf supports both concurrency and request rate. However, we recommend using concurrency. As with request rate, the number of outstanding requests may grow unbounded if the request per second exceeds the system throughput.

When specifying the concurrencies to test, it is useful to sweep over a range of values, from a minimum value of 1 to a maximum value not much greater than the maximum batch size. This is because, when the concurrency is larger than the maximum batch size of the engine, some requests will have to wait in a queue. Therefore, the throughput of the system generally saturates around the maximum batch size while the latency will continue to steadily increase.

Other parameters

In addition, there are relevant LLM serving parameters that can affect the inference performance as well as the accuracy of the benchmark.

Most LLMs have a special end-of-sequence (EOS) token, which signifies the end of the generation. It indicates that the LLM has generated a complete response, and should stop. Under general use, LLM inference should respect this signal and stop generating further tokens. The *ignore_eos* parameter generally instructs whether an LLM inference framework should ignore the EOS token and continue generating tokens until reaching the *max_tokens* limit. For benchmarking purposes, this parameter should be set to True, in order to reach the intended output length and obtain consistent measurement.

Different sampling parameters (like greedy, top_p, top_k, and temperature) might have impacts on the LLM generation speed. Greedy, for example, can be implemented simply by selecting the token with the highest logit. There is no need for normalizing and sorting the probability distribution over tokens, which saves on computation. Whichever sampling method is chosen, it is a good practice to stay consistent within the same benchmarking setup. For a detailed explanation of different sampling methods, see [How to Generate Text: Using Different Decoding methods for Language Generation with Transformers](#).

LLM performance benchmarking is a critical step to ensure both high performance and cost-efficient LLM serving at scale. This post has discussed the most important metrics and parameters when benchmarking LLM inference. To learn more, check out these resources:

- [AI Inference: Balancing Cost, Latency, and Performance](#)
- [How to Deploy NVIDIA NIM in 5 Minutes](#)
- [A Simple Guide to Deploying Generative AI with NVIDIA NIM](#)

Explore the [NVIDIA AI Inference platform](#), and see the latest [AI inference performance data](#). Optimizations from TensorRT, TensorRT-LLM, and TensorRT Model Optimizer libraries are combined and available through production-ready deployments using [NVIDIA NIM microservices](#).

 Discuss (0)

 +29 Like

Tags

[Agentic AI / Generative AI](#) | [Cloud Services](#) | [NIM](#) | [TensorRT-LLM](#) | [Beginner Technical](#) | [Intermediate Technical](#) | [Benchmark](#) | [Deep Dive](#) | [AI Inference](#) | [Featured](#) | [Inference Performance](#) | [LLMs](#) | [Retrieval Augmented Generation \(RAG\)](#)

About the Authors

About Vinh Nguyen

Vinh Nguyen is a deep learning engineer and data scientist, having published more than 50 scientific articles that collectively attracted more than 5,000 citations. At NVIDIA, his work spans a wide range of deep learning and AI applications, including large language models and multi-modality models.

View all posts by Vinh Nguyen >

focusing on LLM training framework and microservices. Her past experience include LLM inference (NIM) and recommender systems (Merlin). She holds a B.S. in computer science from the University of Toronto and an M.B.A. from the MIT Sloan School of Management.

[View all posts by Wenwen Gao >](#)

About Emily Apsey

Emily Apsey leads the NVIDIA virtual GPU Technical Marketing Engineering team. She works closely with technology from original equipment manufacturers, independent software vendors, and hypervisor partners to understand the performance and scalability of applications in virtualized environments.

[View all posts by Emily Apsey >](#)

About Ganesh Kudleppanavar

Ganesh Kudleppanavar is a System Software Manager at NVIDIA, dedicated to optimizing the performance of Machine Learning and Generative AI models. He leads the Triton tools team, utilizing the powerful Triton Tools to meticulously benchmark these models, ensuring their efficient deployment and seamless utilization across diverse applications. Ganesh holds a Master of Science degree in Electrical Engineering from California State University, Long Beach.

[View all posts by Ganesh Kudleppanavar >](#)

About Neelay Shah

Neelay Shah is the principal software architect for NVIDIA Triton Inference Server and an AI solutions engineer. His focus is on enabling developers with a smooth transition from prototyping to high-performance production deployments at scale. Before joining NVIDIA, Neelay was a principal engineer at Intel leading open source projects for computer vision pipelines. He has a bachelor's degree in computer science from Williams College and a master's degree in computer science from UIUC.

About Elias Bermudez

Elias Bermudez is a Senior System Software Engineer at NVIDIA leading the development of GenAI-Perf for the Triton Tools Team. He specializes in providing tools to profile inference performance of AI models. He cares deeply about connecting users with easy to use and powerful software, always iterating to achieve that goal. He holds a Masters of Engineering and BS from Cornell University in Electrical and Computer Engineering as well as a BS in Computer Science from Portland State University.

View all posts by Elias Bermudez [>](#)

Comments

Start the discussion at forums.developer.nvidia.com

Related posts

LLM Inference Benchmarking: How Much Does Your LLM Inference Cost?

Optimizing Inference Efficiency for LLMs at Scale with NVIDIA NIM Microservices

Achieving High Mixtral 8x7B Performance with NVIDIA H100 Tensor Core GPUs and NVIDIA TensorRT-LLM

The Next Generation of AI Begins

It all starts here. June 1-4 | Taipei

[Register Now](#)